Submission Worksheet

Submission Data

Course: IT114-450-M2025
Assignment: IT114 Milestone 1
Student: Preet S. (ps55)
Status: Submitted | **Worksheet Progress:** 95%
Potential Grade: 9.72/10.00 (97.20%)
Received Grade: 0.00/10.00 (0.00%)
Started: 7/8/2025 10:53:12 PM
Updated: 7/10/2025 12:09:19 AM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-1/grading/ps55
View Link: https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-1/view/ps55

Instructions

- Overview Link: https://youtu.be/9dZPFwi76ak

1. Refer to Milestone1 of any of these docs:
    2. Rock Paper Scissors
    3. Basic Battleship
    4. Hangman / Word guess
    5. Trivia
    6. Go Fish
    7. Pictionary / Drawing
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Copy `Part5` and rename the copy as `Project` (this new folder should be in the root of your repo)
6. Organize the files into their respective packages `Client`, `Common`, `Server`, `Exceptions`
    1. Hint: If it's open, you can refer to the Milestone 2 Prep lesson
7. Fill out the below worksheet
    1. Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    2. Since this Milestone was majorly done via lessons, the required comments should be placed in areas of analysis of the requirements in this worksheet. There shouldn't need to be any actual code changes beyond the restructure.
8. Once finished, click "Submit and Export"
9. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `main`
10. Upload the same PDF to Canvas
11. Sync Local

1. `git checkout main`
2. `git pull origin main`

Section #1: (1 pt.) Feature: Server Can Be Started Via Command Line And Listen To Connections

Progress: 100%

Task #1 (1 pt.) - Evidence

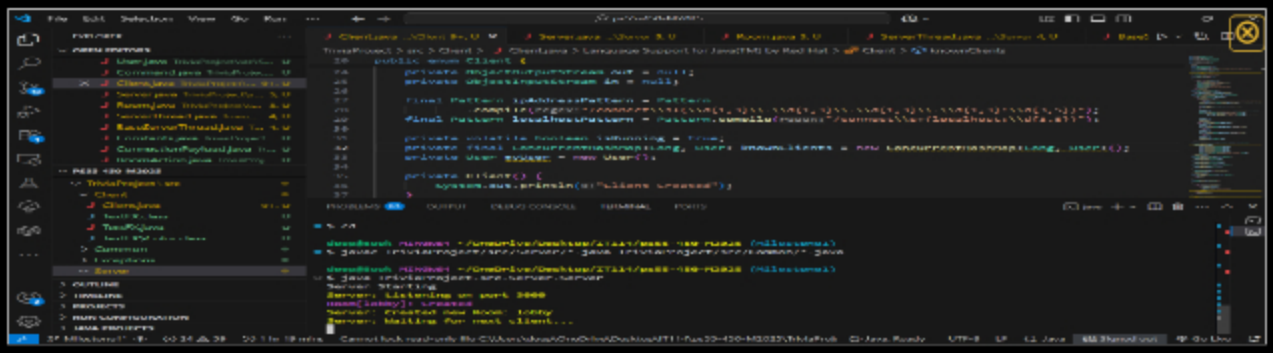
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the server started and listening
- Show the relevant snippet of the code that waits for incoming connections



output

```
try (ServerSocket serverSocket = new ServerSocket(port)) {
    createRoom(Room.LOBBY); // Create the default Lobby

    while (isRunning) {
        info(message:"Waiting for next client...");
        Socket incomingClient = serverSocket.accept();
        info(message:"Client connected!");

        ServerThread serverThread = new ServerThread(incomingClient, this::onServerThreadInitialize
        serverThread.start();
    }

} catch (DuplicateRoomException e) {
    System.err.println(TextFX.colorize(text:"Lobby already exists (shouldn't happen)", Color.RED));
} catch (IOException e) {
    System.err.println(TextFX.colorize(text:"Error accepting connection", Color.RED));
    e.printStackTrace();
} finally {
    info(message:"Closing server socket");
```

snippet

Saved: 7/9/2025 11:44:33 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side waits for and accepts/handles connections

Your Response:

The server starts from the command line with a ServerSocket that listens on a specified port. It waits for connections using a blocking accept() call inside a loop. Each new client connection is handled in its own ServerThread, allowing the server to accept multiple clients at once while continuing to listen for more.

Saved: 7/9/2025 11:44:33 PM

Section #2: (1 pt.) Feature: Server Should Be Able To Allow More Than One Client To Be Connected At Once

Progress: 100%

Task #1 (1 pt.) - Evidence

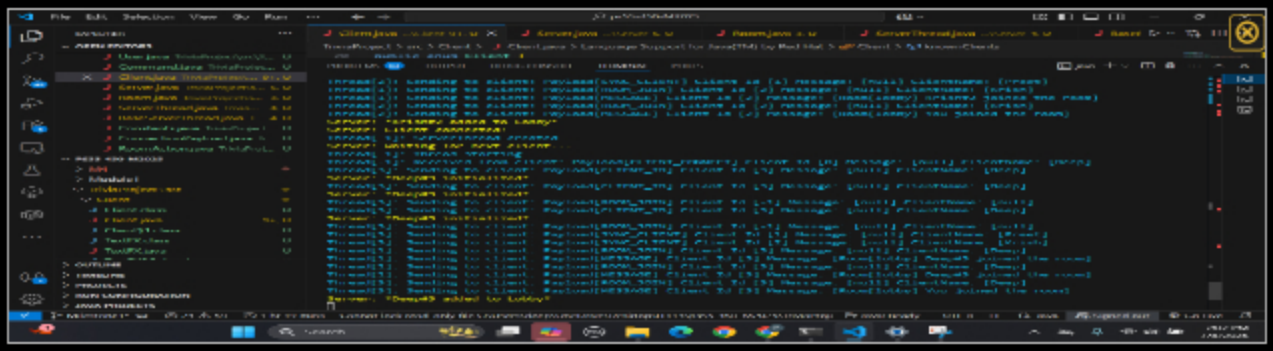
Progress: 100%

Part 1:

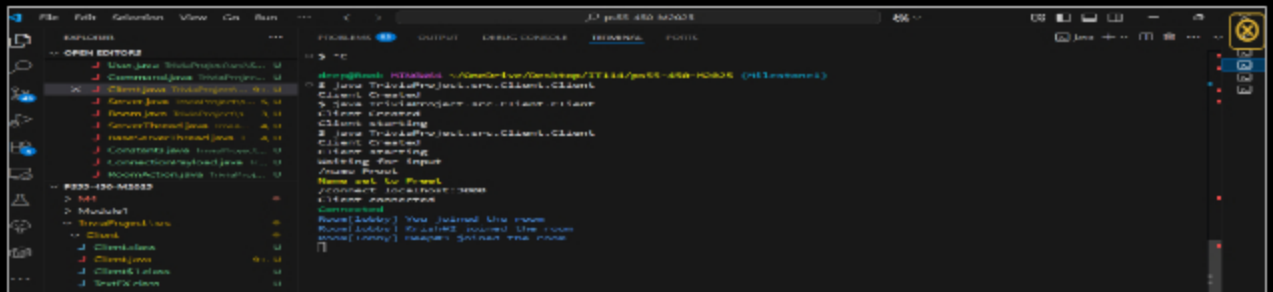
Progress: 100%

Details:

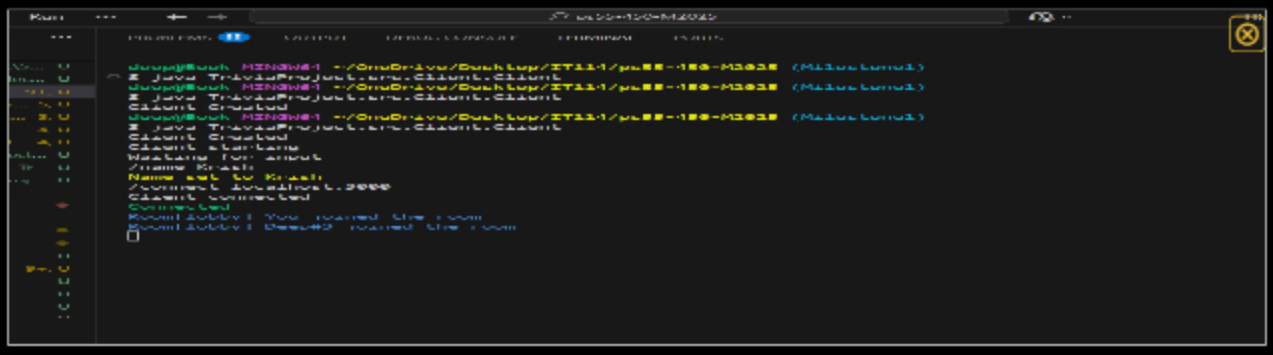
- Show the terminal output of the server receiving multiple connections
- Show at least 3 Clients connected (best to use the split terminal feature)
- Show the relevant snippets of code that handle logic for multiple connections



Server receiving multiple connections

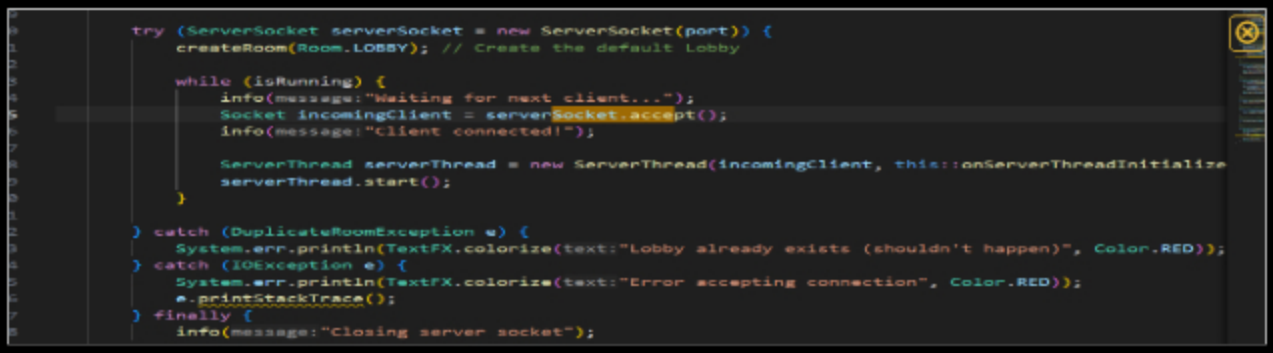


client1



client2

client3



snippet

Saved: 7/9/2025 11:42:54 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles multiple connected clients

Your Response:

The server handles multiple clients by creating a new ServerThread for each connection. Each ServerThread runs in parallel on its own thread, so clients can connect, send, and receive messages independently while the server keeps listening for more connections.

Saved: 7/9/2025 11:42:54 PM

Section #3: (2 pts.) Feature: Server Will Implement The Concept Of Rooms (With The Default Being "l obby")

Progress: 100%

≡ Task #1 (2 pts.) - Evidence

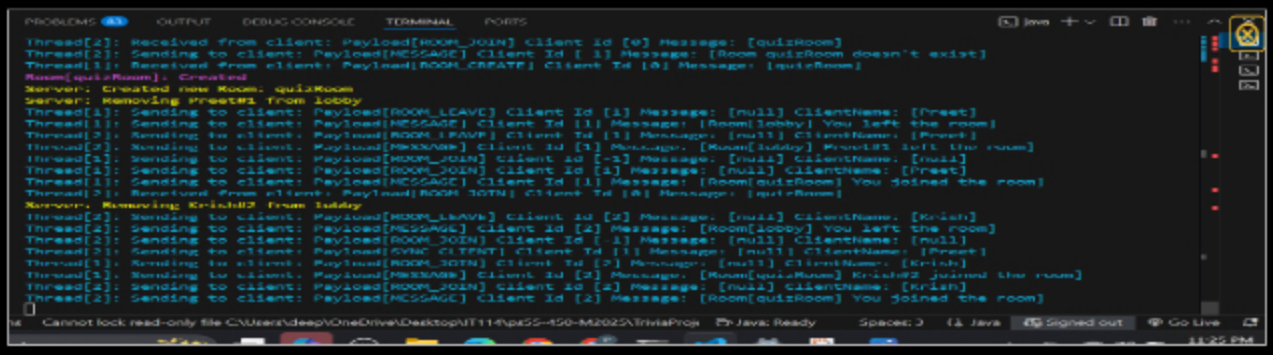
Progress: 100%

Part 1:

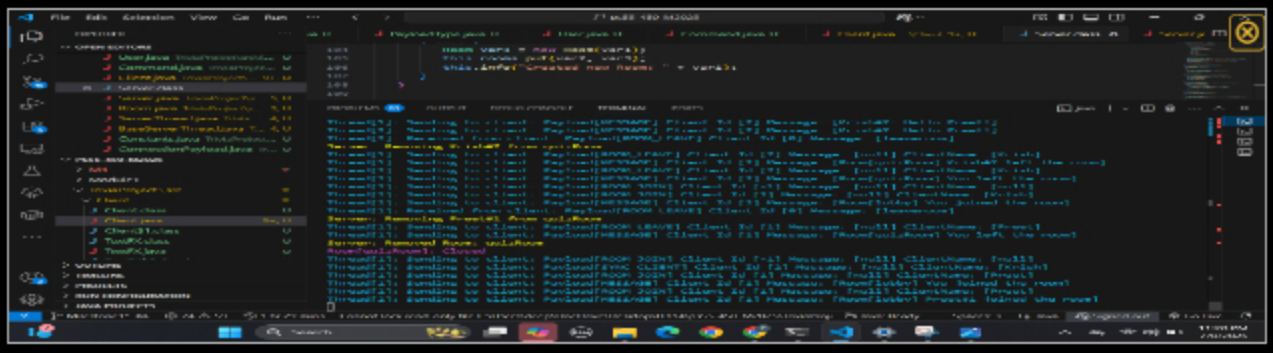
Progress: 100%

Details:

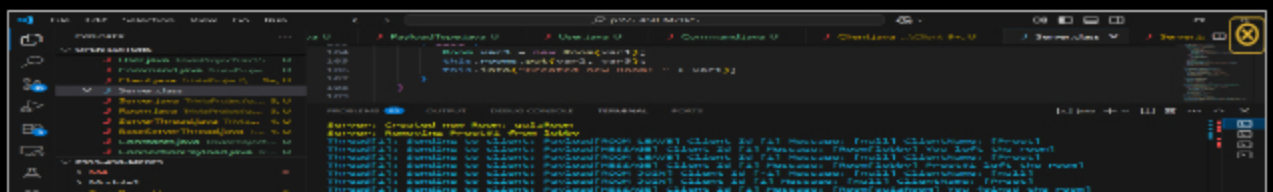
- Show the terminal output of rooms being created, joined, and removed (server-side)
- Show the relevant snippets of code that handle room management (create, join, leave, remove) (server-side)

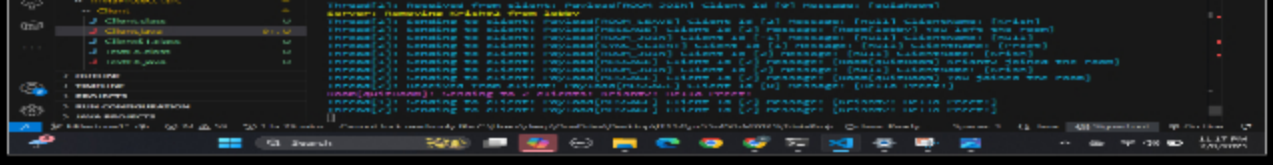


creating and joining room

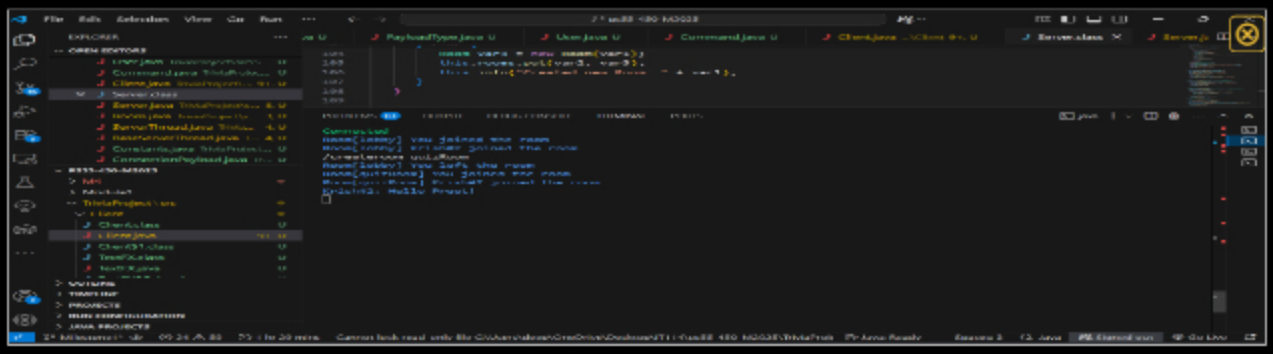


removing room

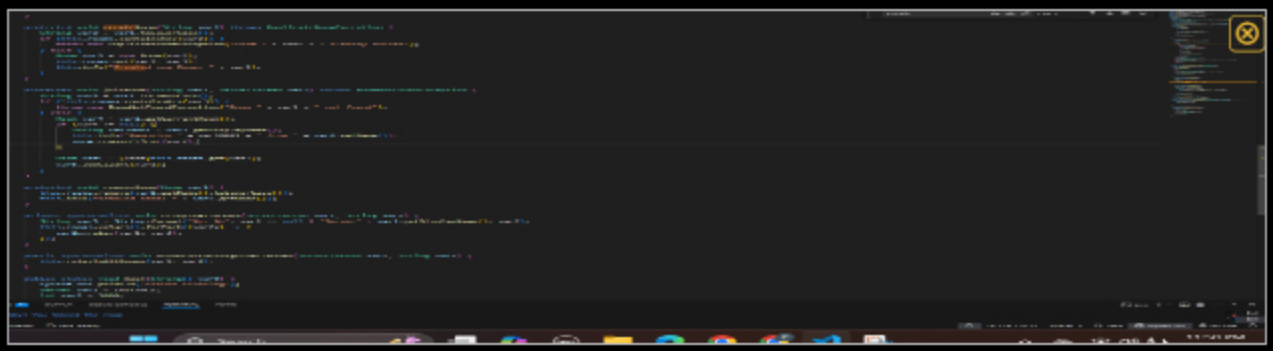




creating and joining room



client to client msg



snippet

Saved: 7/10/2025 12:07:19 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles room creation, joining/leaving, and removal

Your Response:

The server always creates a default “lobby” room when it starts. Clients can create new rooms, which are added to the server’s room map.When a client joins or leaves a room, the server moves them by updating their ServerThread’s current room. If a room becomes empty (except the lobby), the server automatically removes it to free up resources.

Saved: 7/10/2025 12:07:19 AM

Section #4: (1 pt.) Feature: Client Can Be

Started Via The Command Line

Progress: 100%

Task #1 (1 pt.) - Evidence

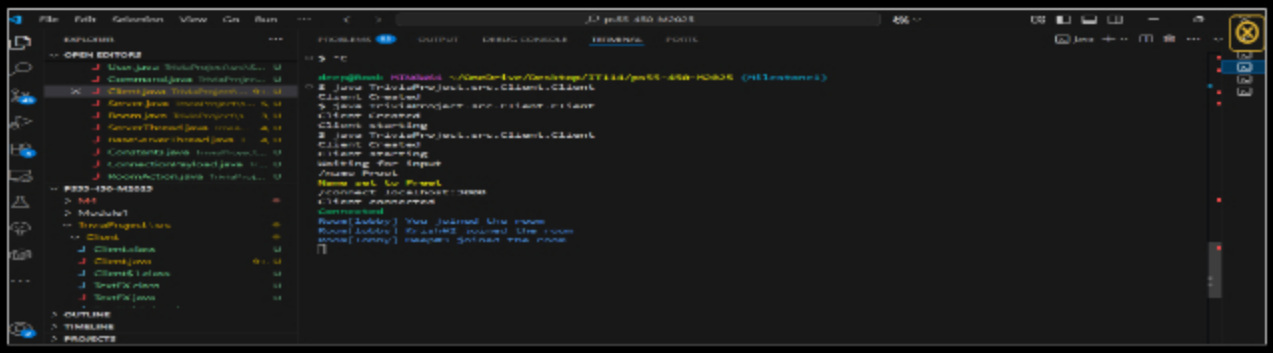
Progress: 100%

Part 1:

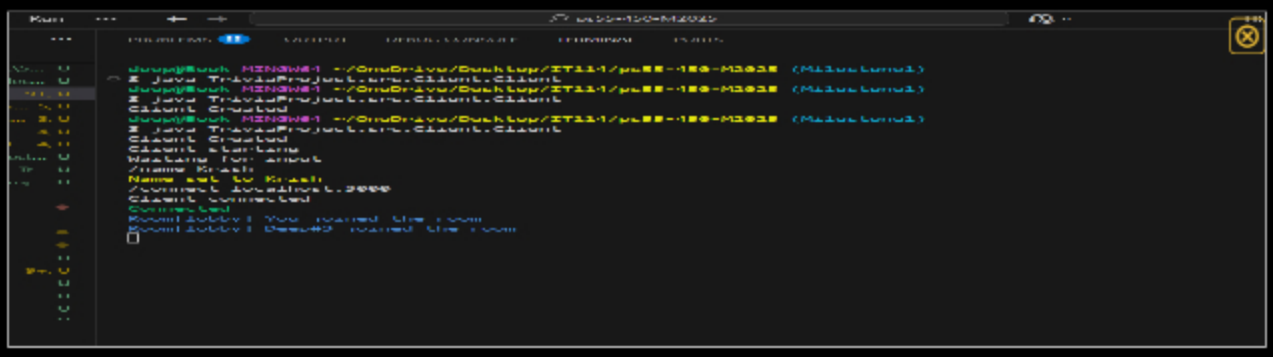
Progress: 100%

Details:

- Show the terminal output of the /name and /connect commands for each of 3 clients (best to use the split terminal feature)
- Output should show evidence of a successful connection
- Show the relevant snippets of code that handle the processes for /name, /connect, and the confirmation of being fully setup/connected

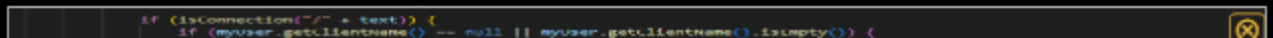


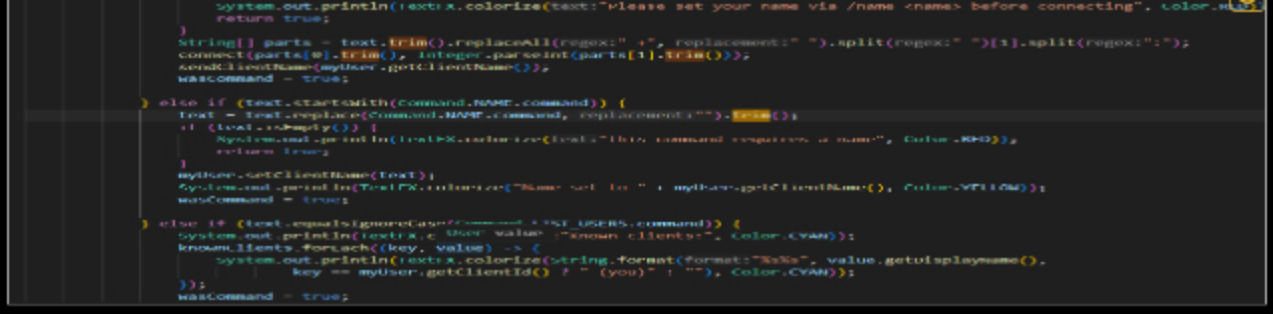
client1



client2

client3





snippet

Saved: 7/10/2025 12:03:25 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how the /name and /connect commands work and the code flow that leads to a successful connection for the client

Your Response:

The client starts from the command line and uses /name to set its name locally.The /connect command connects to the server socket and sends the client name as part of a handshake.When the server replies with the client ID, the client confirms it's fully connected and ready.

Saved: 7/10/2025 12:03:25 AM

Section #5: (2 pts.) Feature: Client Can Create/j oin Rooms

Progress: 100%

Task #1 (2 pts.) - Evidence

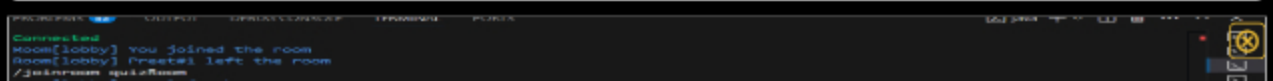
Progress: 100%

Part 1:

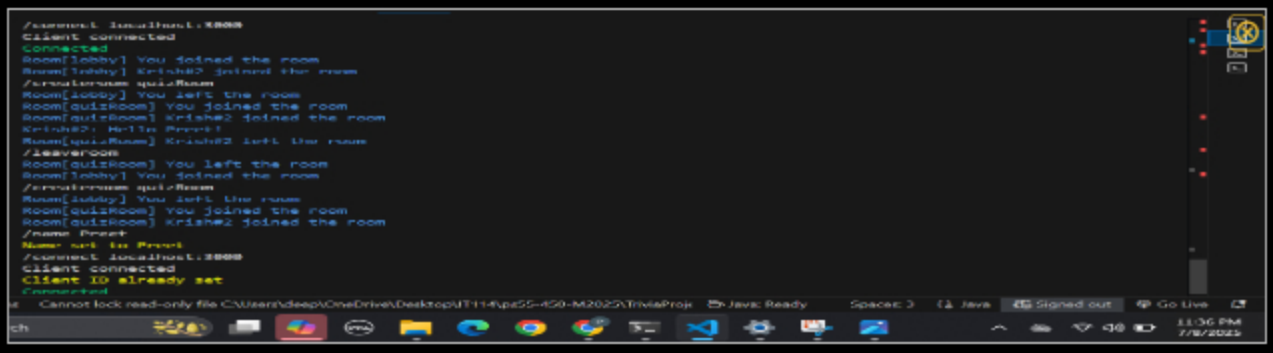
Progress: 100%

Details:

- Show the terminal output of the /createroom and /joinroom
- Output should show evidence of a successful creation/join in both scenarios
- Show the relevant snippets of code that handle the client-side processes for room creation and joining

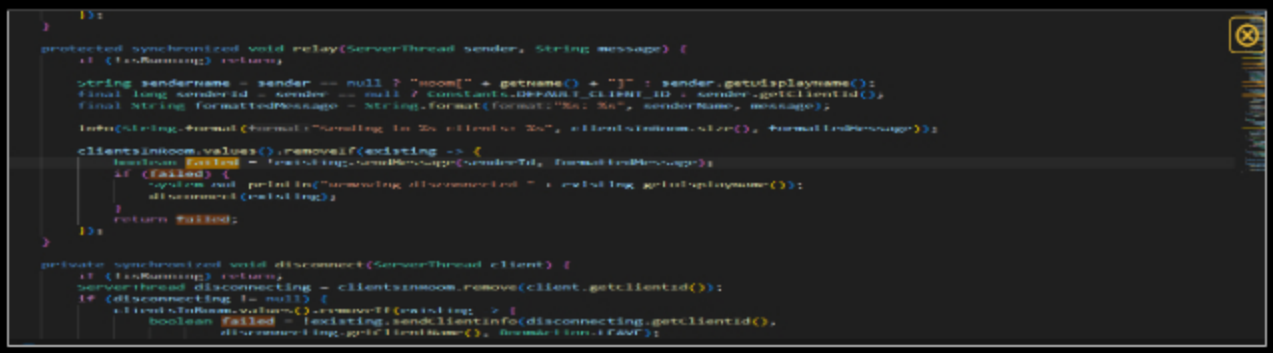


joining output



create/leave output

snippet



snippet

Saved: 7/10/2025 12:06:50 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how the /createroom and /join room commands work and the related code

flow for each

Your Response:

The /createroom command lets a client create a new room by sending a special payload to the server, which adds the room and moves the client into it. The /joinroom command lets a client switch rooms by telling the server which room to join; the server removes the client from their current room and adds them to the target room, updating other clients in both rooms.

Saved: 7/10/2025 12:06:50 AM

Section #6: (1 pt.) Feature: Client Can Send Messages

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

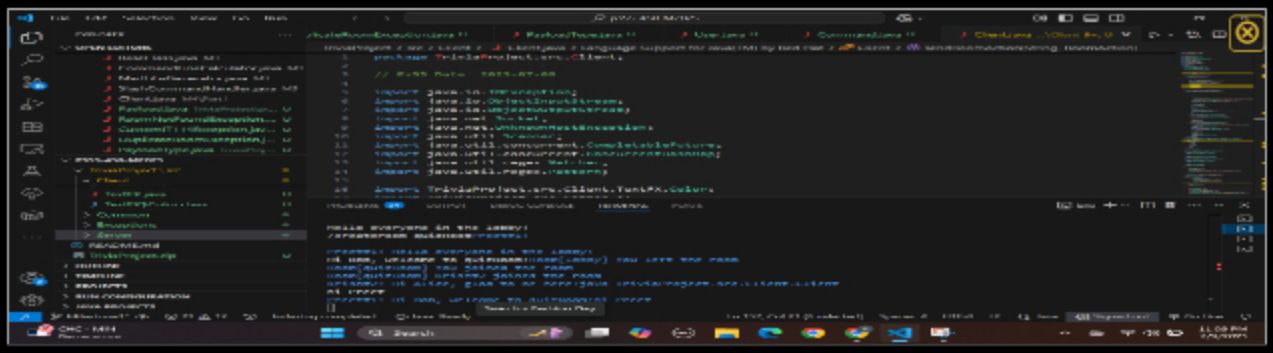
Part 1:

Progress: 100%

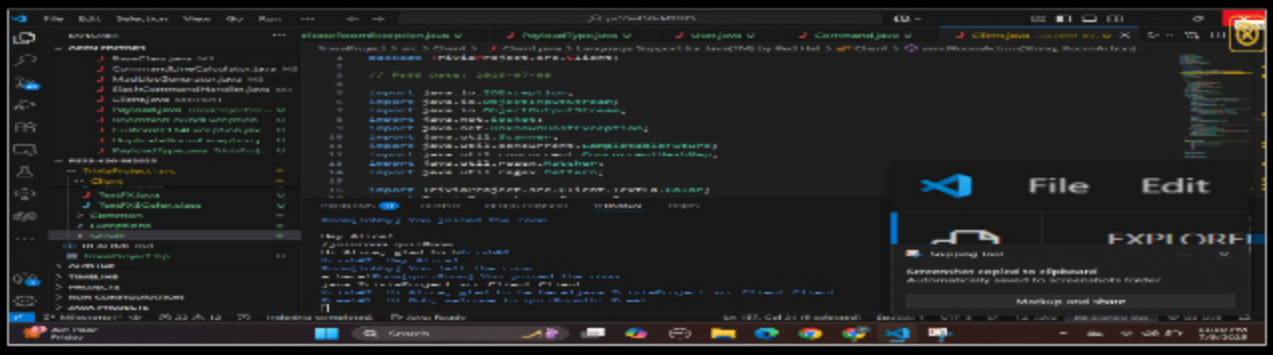
Details:

- Show the terminal output of a few messages from each of 3 clients
- Include examples of clients grouped into other rooms
- Show the relevant snippets of code that handle the message process from client to server-side and back

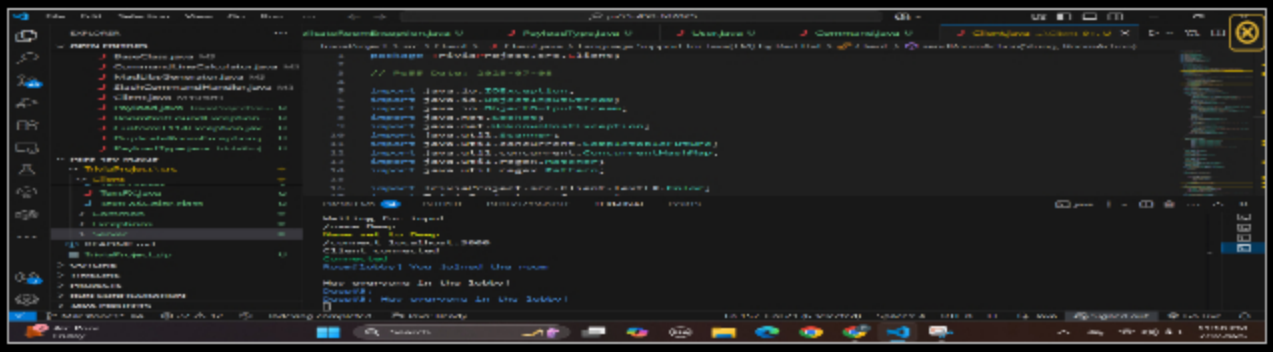
server



client1



client2



client3

```
protected void processPayload(Payload incoming) {
    switch (incoming.getPayloadType()) {
        case CLIENT_CONNECT:
            setClientName(((ConnectionPayload) incoming).getClientName().trim());
            break;

        case DISCONNECT:
            if (currentRoom != null) {
                currentRoom.handleDisconnect(this);
            }
            break;

        case MESSAGE:
            if (currentRoom != null) {
                currentRoom.handleMessage(this, incoming.getMessage());
            }
            break;

        case REVERSE:
            if (currentRoom != null) {
```

snippet

```
            } else if (text.startsWith(Command.REVERSE.command)) {
                text = text.replace(Command.REVERSE.command, replacement: "").trim();
                sendReverse(text);
                wasCommand = true;

            } else if (text.startsWith(Command.CREATE_ROOM.command)) {
                text = text.replace(Command.CREATE_ROOM.command, replacement: "").trim();
                if (text.isEmpty()) {
                    System.out.println(Textfx.colorize(text: "This command requires a room name", Color.RED));
                    return true;
                }
                sendRoomAction(text, RoomAction.CREATE);
                wasCommand = true;

            } else if (text.startsWith(Command.JOIN_ROOM.command)) {
                text = text.replace(Command.JOIN_ROOM.command, replacement: "").trim();
                if (text.isEmpty()) {
                    System.out.println(Textfx.colorize(text: "This command requires a room name", Color.RED));
                    return true;
                }
                sendRoomAction(text, RoomAction.JOIN);
                wasCommand = true;

            } else if (text.startsWith(Command.LEAVE_ROOM.command)) {
                sendRoomAction(text, RoomAction.LEAVE);
                wasCommand = true;
            }
```

snippet

Saved: 7/10/2025 12:06:28 AM

Part 2:

Progress: 100%

Details:

- Briefly explain how the message code flow works

Your Response:

When a client types a message, it's wrapped in a Payload with type MESSAGE and sent to the server. The server's ServerThread receives it and passes it to the client's current Room. The Room uses relay() to forward that message to all other clients in the same room only, so messages stay private to each room and don't leak to other rooms.

Saved: 7/10/2025 12:06:28 AM

Section #7: (1 pt.) Feature: Disconnection

Progress: 87%

Task #1 (1 pt.) - Evidence

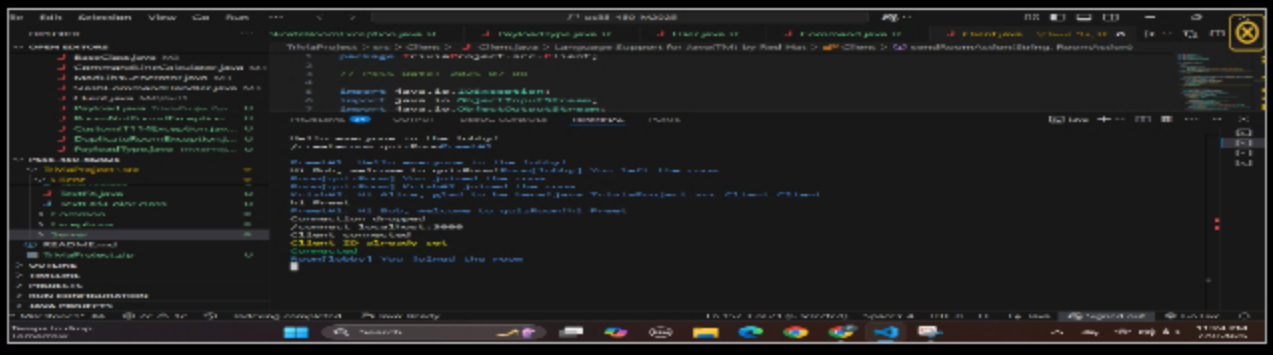
Progress: 87%

Part 1:

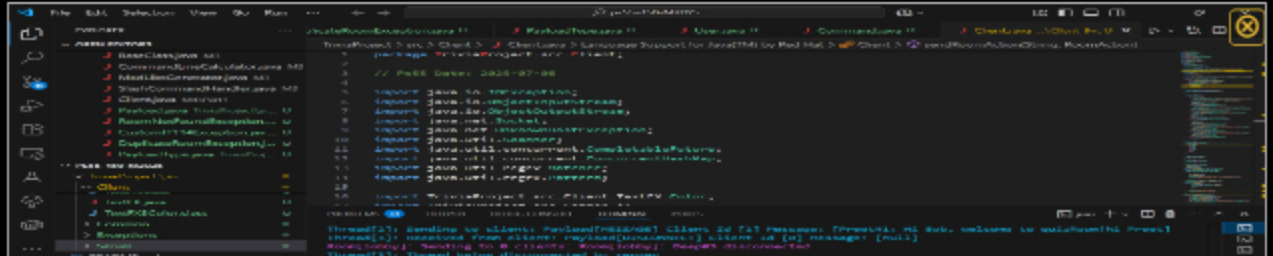
Progress: 75%

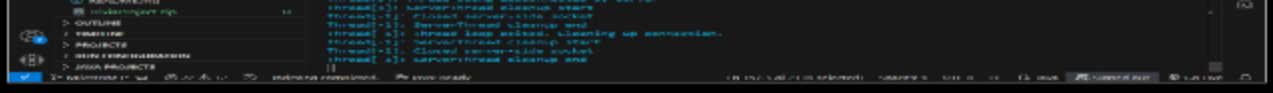
Details:

- Show examples of clients disconnecting (server should still be active)
- Show examples of server disconnecting (clients should be active but disconnected)
- Show examples of clients reconnecting when a server is brought back online
- Examples should include relevant messages of the actions occuring
- Show the relevant snippets of code that handle the client-side disconnection process
- Show the relevant snippets of code that handle the server-side termination process

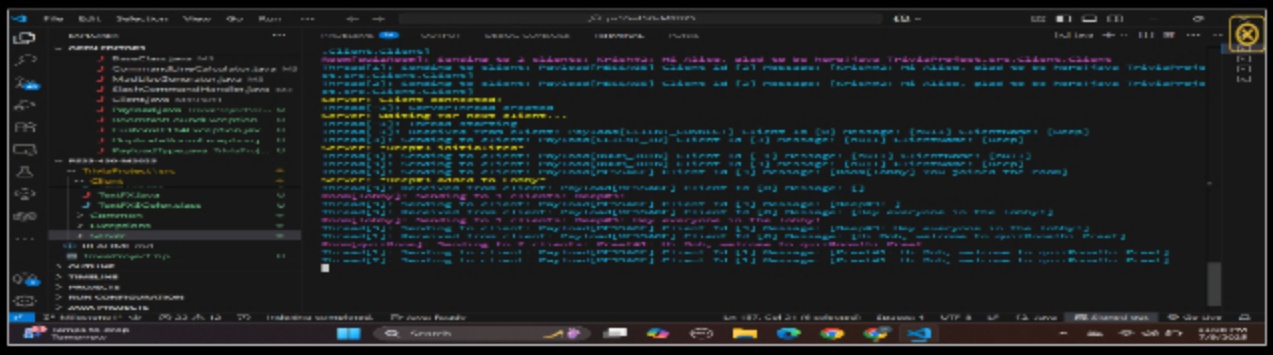


reconnect





disconnect



server disconnect

Missing Caption

Saved: 7/9/2025 11:31:33 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how both client and server gracefully handle their disconnect/termination logic

Your Response:

The client handles disconnects by sending a special DISCONNECT payload to the server, which removes the client from its current room and closes its ServerThread cleanly. If the server shuts down, it uses a shutdown hook to disconnect all connected clients and free up resources. Clients stay open, detect the lost connection, and can reconnect when the server comes back online.

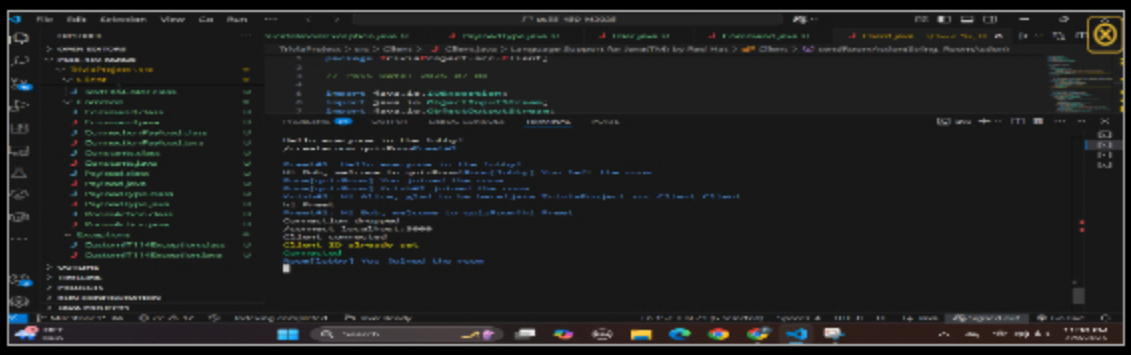
Saved: 7/9/2025 11:31:33 PM

Section #8: (1 pt.) Misc

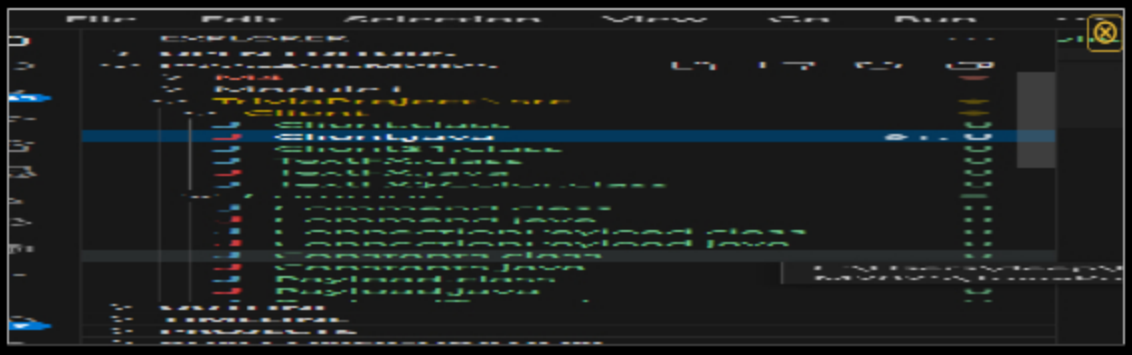
Progress: 87%

Task #1 (0.25 pts.) - Show the proper workspace structure with the new Client, Common, Server, and Exceptions packages

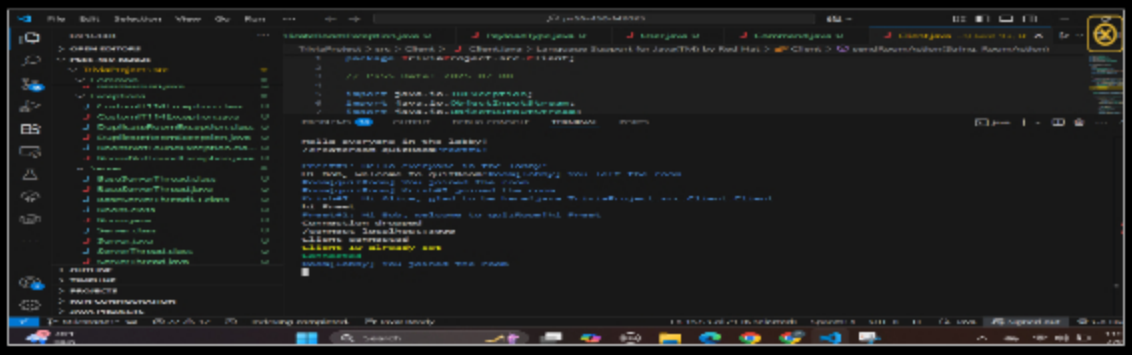
Progress: 100%



structure1



structure2



structure3

Saved: 7/9/2025 11:32:20 PM

Task #2 (0.25 pts.) - Github Details

Progress: 50%

Part 1:

Progress: 0%

Details:
From the Commits tab of the Pull Request screenshot the commit history

Missing Caption

Saved: 7/10/2025 12:07:57 AM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/preets4001/ps55-450-M2025/pulls

URL
https://github.com/preets4001/ps

Saved: 7/10/2025 12:07:57 AM

Task #3 (0.25 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

wakatime

Saved: 7/9/2025 11:34:43 PM

Task #4 (0.25 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Through this milestone, I learned how to organize a multi-package Java project by separating the client, server, common logic, and custom exceptions in a clear workspace structure. I also practiced using sockets to handle multiple client connections at the same time with dedicated threads, and saw how rooms help manage group communication.

Saved: 7/9/2025 11:35:56 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of this assignment was setting up the basic project structure and running the server and client from the command line. Using the lesson code as a starting point made it straightforward to organize the files into the correct packages and test simple commands like /name and /connect. Once the server and client could communicate, it was also pretty easy to see messages relayed in the terminal, which helped confirm that the basic connection logic was working correctly.

Saved: 7/10/2025 12:09:19 AM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of this assignment was making sure the server correctly handled multiple clients at the same time without conflicts, especially when clients switched rooms or disconnected. It was also challenging to test all the different scenarios, like reconnecting after a server shutdown, and making sure each client stayed isolated in the right room. Debugging the socket connections and thread cleanup took extra effort but really helped me understand how multi-threaded networking works.

Saved: 7/9/2025 11:36:25 PM